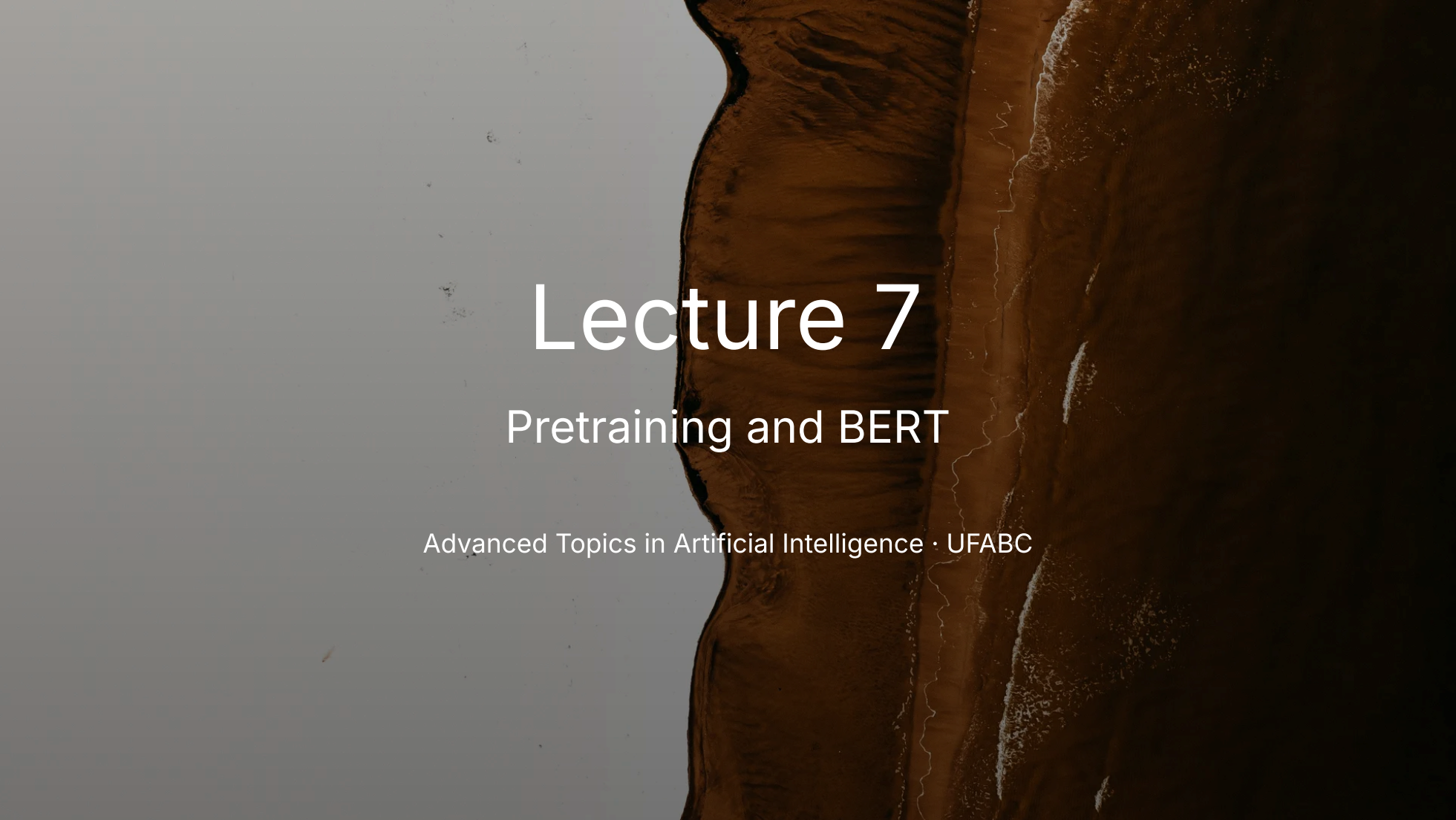




Lecture 7

Pretraining and BERT

Advanced Topics in Artificial Intelligence · UFABC

Lecture outline

Part 1 — Representations as Knowledge Neural networks as representation learners

Part 2 — Self-Supervised Learning Creating labels from the data itself

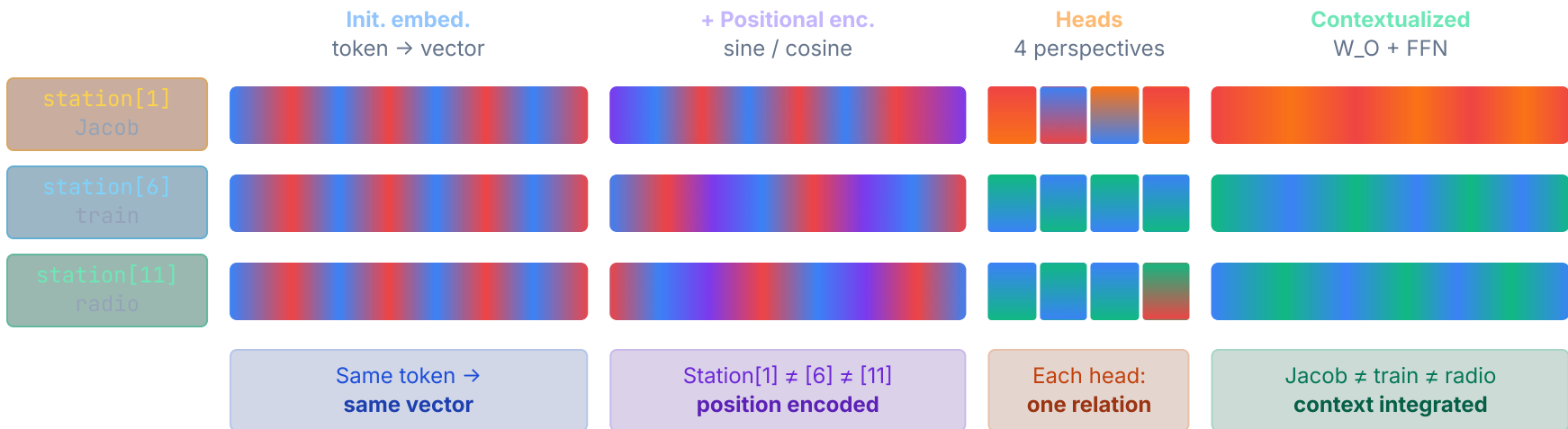
Part 3 — BERT Architecture, pretraining, and fine-tuning

Part 4 — HuggingFace Hub Ecosystem of pretrained models

Part 5 — Transformers Beyond NLP ViT, TabTransformer, and multimodality

The Transformer transforms embeddings

"Jacob **Station** lives nearby the train **station** close to the radio **station**"



illustrative bars — real values visualized in the notebook

Part 1 — Representations as Knowledge

Neural networks learn representations

Each layer's output is a representation

The original input is progressively transformed at every hidden layer.

Rather than a "final answer," each layer produces a new view of the data.

Visual hierarchy in CNNs

Early layers detect **lines and edges**

Middle layers detect **shapes and textures**

Deep layers detect **complex objects**

Final layer detects **semantic categories**

Intuition: A network trained to classify 1000 ImageNet categories did not memorize pixels — it learned a hierarchy of visual concepts. The intermediate representations encode general knowledge about the visual world.

Transfer Learning: reusing representations

Classic scenario

1. Train ResNet on ImageNet (1.2M images, 1000 classes)
2. Remove the output layer
3. Use the rest as a **fixed encoder**
4. Add a new head for the target task

Why does it work?

With only 100 examples of bags vs shoes, the encoder already captured:

- shapes and contours
- textures and materials
- object parts

Transfer Learning flow

```
ResNet (pretrained on ImageNet)
    ↓
[remove last layer]
    ↓
Representation encoder
    ↓
Dense → softmax ← train with 100 examples
```

Deep-layer representations generalize to related tasks without needing millions of labeled examples for the target task.

For NLP: we need a text encoder

What we want

An encoder that produces **rich representations** of text: capturing semantics, syntax, context, and relationships between words.

The label problem

Supervised training requires human-annotated labels.

Annotation is expensive and slow — and many tasks have very few examples available.

The good news

Raw text is available in practically unlimited quantities:

- Wikipedia (billions of words)
- Digital books (BooksCorpus, Project Gutenberg)
- Web pages (Common Crawl)

We can exploit this abundance without any human labels.

Part 2 — Self-Supervised Learning

The costly label problem

Traditional supervised learning

Requires manually annotated (input, label) pairs.

Sentiment classification: a human reads a sentence and marks positive/negative.

NER: a human identifies each entity in the text.

Cost: hours of specialist work per thousands of examples.

What we have in abundance

Billions of unlabeled sentences: books, articles, forums, source code.

No human labels — but all the structure of language is there.

Question: can we learn useful representations without any labels?

Self-Supervised Learning: automatically create labels from the structure of the data itself. Use part of the input to predict another part of the same input.

Masking: the central idea

Masking procedure

1. Take original text
2. Replace random tokens with MASK
3. Train model to predict the masked tokens
4. Labels = original tokens (extracted automatically)

No human annotation needed.

Creating training pairs

Masked input → Label = original token

Fully automatic generation from any text corpus.

Concrete example:

Original: "The mission of the MIT Sloan School of Management is to develop principled, innovative leaders..."

Masked: "The MASK of the MIT Sloan School of MASK is to develop principled, innovative MASK..."

Labels: "mission", "Management", "leaders"

Example: filling in the blanks

Input text (with masks):

"The MASK of the MIT Sloan School of MASK is to develop principled, innovative MASK who improve the world and to generate ideas that advance management practice."

MASK 1

To predict "mission," the model must understand that "The ___ of the MIT Sloan School" calls for a noun expressing institutional purpose.

Requires **semantics** and **syntactic structure**.

MASK 2

"MIT Sloan School of ___" — the model must know that schools have full names and that "Management" completes this one.

Requires **world knowledge**.

MASK 3

"develop principled, innovative ___" — the model infers that people (leaders/thinkers) complete the phrase.

Requires **semantic relations**.

Why does masking work?

To fill in blanks, the model must:

- Understand the meaning of surrounding words
- Capture long-range dependencies
- Model semantic and grammatical coherence
- Integrate bidirectional context (before and after)

Result

The internal representations learned during masking automatically encode all this linguistic knowledge — with zero human labels.

As a sequence labeling task:

```
Masked input
  ↓
Transformer Encoder
  ↓
Per-token embedding
  ↓
Dense(ReLU)
  ↓
Softmax → predicted token
```

It is a sequence labeling problem: each masked token has a label (the original token). The model outputs a distribution over the vocabulary for each MASK.

Part 3 — BERT

BERT: overview

BERT

Bidirectional Encoder Representations from Transformers

Devlin et al., 2018 — Google AI

Architecture

Transformer Encoder pretrained with:

- Masked Language Modeling (MLM)
- Next Sentence Prediction (NSP)

Corpus: Wikipedia + BooksCorpus (~3.3B words)

Original variants

BERT-base: 12 layers, 768 dim, 12 heads — 110M parameters

BERT-large: 24 layers, 1024 dim, 16 heads — 340M parameters

Pretraining: weeks on TPUs. Fine-tuning: minutes/hours on a single GPU.

Core idea: pretrain once on massive text, then quickly adapt to any task with a few labeled examples.

BERT is bidirectional — what does that mean?

Unidirectional models (e.g. GPT)

When processing the word "bank" at position 4, the model only sees prior tokens:

She went to the **bank** to deposit money

The context to the **right** is blocked — necessary for text generation, but limiting for comprehension.

Why does MLM allow bidirectionality?

In next-word-prediction (GPT), seeing the future would mean "copying the answer" — the model would simply read the next token directly.

In MLM (BERT), the target token is **absent** from the input ([MASK]). Using context from both sides to predict the missing token is entirely legitimate — that is exactly what we want.

BERT: full bidirectional attention

When encoding "bank", the model sees **all** tokens:

She went to the **bank** to deposit money

Attention mask: the visual difference

GPT (causal)



BERT (bidirectional)



BERT Architecture: encoder stack

[CLS] tok₁ tok₂ ... tok_n [SEP]

Input embedding

Token emb. + Segment emb. + Position emb. (learnable)
→ element-wise sum → dim 768

Encoder Block × 12 (base) / × 24 (large)

- ① Multi-Head Self-Attention 12 heads, d_k = 64
- ② Add & LayerNorm residual + norm
- ③ FFN: Linear → GeLU → Linear 768 → 3072 → 768
- ④ Add & LayerNorm residual + norm

Encoder Block × 12 ...

Encoder Block × 12 ...

h_{CLS} h₁ h₂ ... h_n (dim 768 each)

BERT-base vs BERT-large

Config	base	large
Layers (L)	12	24
Hidden size (H)	768	1024
Heads (A)	12	16
FFN dim	3072	4096
Parameters	110M	340M

Three embeddings summed at input

Token: lookup — 30k-token table (WordPiece)

Segment: A or B — sentence pairs

Position: learnable (≠ sine/cosine of original Transformer)

Why GeLU in the FFN? Smoother gradients than ReLU; empirically converges better in LLMs.

Fine-tuning BERT: strategies and hyperparameters

Strategy 1 — Frozen encoder

Freeze BERT, train only the classification head. Fast, works with dozens of examples; does not adapt representations.

Strategy 2 — Full fine-tuning

Update all weights with low lr ($\sim 2e-5$). Best performance, but high lr causes **catastrophic forgetting**.

Strategy 3 — Layer-wise LR decay

Deeper layers = smaller lr; output layers = larger lr. Preserves general representations, adapts the top.

Recommended hyperparameters (Devlin et al., 2018)

lr:	2e-5, 3e-5, 5e-5
batch size:	16, 32
epochs:	2-4
warmup:	10%
weight decay:	0.01

When to use each strategy?

Available data	Strategy
< 100 examples	Frozen encoder
100-5k	Full fine-tuning
> 5k	Full + layer-wise decay

The CLS token: global representation

Special token CLS

Added at the beginning of every input sequence.

During pretraining, the CLS embedding learns to aggregate information from the entire sequence via self-attention.

By the end of the network, it represents the sentence as a whole — global context.

BERT input structure

CLS] token1 token2 ... tokenN [SEP

↓ ↓ ↓ ↓
h_CLS h_1 h_2 ... h_N

h_CLS: global sentence embedding

h_i: contextualized embedding of token i

Why does CLS work as a global representation?

In the Transformer, all tokens communicate via self-attention at every layer. CLS carries no lexical meaning of its own, so it learns to aggregate information from all other tokens to fulfill the pretraining objective (NSP). After fine-tuning, it carries the semantic representation of the whole sentence.

Sentence-BERT: beyond CLS

The [CLS] embedding is great for classification, but **not ideal for semantic sentence similarity**.

Limitation of CLS for similarity

Comparing N sentences requires N^2 inferences. Also, cosine distance between [CLS] embeddings of independent sequences **does not correlate well** with human semantic similarity.

Solution: Sentence-BERT (Reimers & Gurevych, 2019)

BERT trained with a **siamese network** on similarity-labeled pairs. Uses **mean pooling** instead of [CLS] .

Result: N inferences, not N^2 . Embeddings comparable by cosine.

Siamese architecture:

```
sentence A → BERT → mean pool → u ┐  
                                     │  
sentence B → BERT → mean pool → v ┘ concat(u, v, |u-v|) → similarity  
      (shared weights – trained with NLI)
```

Fine-tuning for sequence classification

Task: sentiment analysis

Given a sentence, classify it as positive or negative.

Fine-tuning architecture

Input: [CLS] I loved this movie [SEP]

↓

BERT Encoder (pretrained)

↓

[CLS] embedding: h_{CLS}

↓

Dense(ReLU)

↓

sigmoid / softmax

↓

positive / negative

Fine-tuning strategies

Head-only training: freeze BERT, train only Dense + softmax. Fast, works well with few examples.

Full fine-tuning: update all BERT weights together with the head. Better performance, needs more data and time.

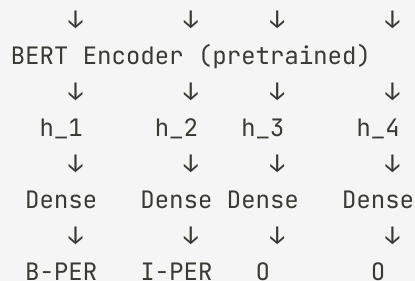
Fine-tuning for sequence labeling

Tasks: NER, slot filling, POS tagging

Each token receives a class label (e.g., person, organization, location, other).

Fine-tuning architecture

CLS] Barack Obama was president [SEP



BERT's flexibility

The same pretrained architecture handles both task types. Only the head added during fine-tuning changes:

- Classification: use h_{CLS}
- Labeling: use h_i for each token

BERT variants

RoBERTa (Liu et al., 2019 — Facebook)

Improvements over BERT:

- Trained longer
- Larger batch (8k samples)
- No NSP objective
- More training data

Better performance on NLU benchmarks.

DistilBERT (Sanh et al., 2019 — HuggingFace)

Knowledge distillation from BERT:

- 40% smaller
- 60% faster
- 97% of BERT's performance

Ideal for production with resource constraints.

Multilingual BERT (Google, 2018)

- 104 languages
- Single model
- Trained on multilingual Wikipedia

Works for Portuguese, Spanish, Chinese, Arabic, etc. — no per-language fine-tuning needed.

Sentence Transformers (sbert.net): BERT variants optimized for generating semantically similar sentence embeddings. Use contrastive training with sentence pairs. Ideal for semantic search, deduplication, and clustering.

Part 4 — HuggingFace Hub

HuggingFace: the model ecosystem

The Hub

500k+ pre-trained models, datasets and demo spaces — NLP, vision, audio, multimodal, dozens of languages. Anyone can publish and download.

Typical workflow

1. Choose a model on the Hub for your task
2. Load with `AutoTokenizer` / `AutoModel`
3. Add a task head (classification, NER...)
4. Fine-tune with a few labeled examples

Key concept

Use BERT (or a variant) as a pre-trained encoder and train only the output head with your examples. No need for millions of data points — BERT already knows about language.

Code: HuggingFace Transformers

```
from transformers import pipeline

# Sentiment analysis with BERT
classifier = pipeline("sentiment-analysis")
result = classifier("I loved the movie!")
# [{'label': 'POSITIVE', 'score': 0.9998}]

# Fill-mask
fill = pipeline("fill-mask", model="bert-base-uncased")
fill("The capital of France is [MASK].")
# [{'token_str': 'paris', 'score': 0.9975}]

# NER
ner = pipeline("ner", aggregation_strategy="simple")
ner("Hugging Face is based in New York")
```

pipeline() automatically loads the model + tokenizer + post-processing for the chosen task. Just pass the task name.

AutoTokenizer / AutoModelForSequenceClassification
allow loading any model from the Hub and adding custom heads for full fine-tuning.

Part 5 — Transformers

Beyond NLP

Vision Transformer (ViT)

Idea: treat an image as a sequence of patches

1. Divide image into fixed-size patches (e.g., 16×16 pixels)
2. Flatten each patch into a vector
3. Linear projection $\rightarrow$ patch embedding
4. Add positional encoding
5. Feed into Transformer Encoder
6. Use CLS for classification

Result

Dosovitskiy et al., 2020 (Google Brain):

With sufficient pretraining data scale, ViT is competitive with or superior to CNNs for image classification.

ViT flow:

```
224x224 image
  ↓
Split into 16x16 patches → 196 patches
  ↓
Flatten + Linear Projection → 196 embeddings
  ↓
[CLS] + positional encoding
  ↓
Transformer Encoder (same as in NLP)
  ↓
h_CLS → Dense → image class
```

TabTransformer: tabular data

The tabular data problem

Categorical variables have complex interactions that linear models fail to capture.

Tree-based models capture interactions but do not generalize well with little data.

TabTransformer architecture (Huang et al., 2020)

Categorical variables



Per-column embeddings



Transformer Encoder
(captures column interactions)



Numerical variables → concatenate



Dense layers → prediction

Advantage: the Transformer automatically discovers which combinations of columns matter for prediction — something manual feature engineering would try to replicate. Especially useful when categorical variables interact in non-obvious ways.

Embeddings: the common language

Text

Tokenize → per-token embedding

The same Transformer Encoder processes the sequence of embeddings.

Image

Split into patches → linear projection

Patches become tokens — same Transformer.

Table / Audio / Graph

Any data can be tokenized and projected into embeddings of the same dimension.

The unifying power of embeddings

The Transformer Encoder is domain-agnostic — it only needs a sequence of fixed-dimension vectors. What changes across domains is only the tokenization/projection module.

This enables **multimodal** models: combining text + image + table in the same architecture, mixing embeddings from different domains in the same sequence. CLIP, Flamingo, and GPT-4V are examples of this paradigm.

Summary: what we learned today

Representations as knowledge

Deep neural networks learn representation hierarchies.
Transfer learning reuses that knowledge for new tasks.

Self-Supervised Learning

Artificial labels created from the data. Masking forces the model to learn language without human supervision.

BERT and fine-tuning

Encoder pre-trained on massive text. `[CLS]` for classification; `h_i` per token for sequence labeling.

HuggingFace

500k+ ready models. Pipeline in 3 lines: choose model → tokenizer → head → fine-tune.

Transformers beyond NLP

ViT (images as patches), TabTransformer (columns as tokens). Embeddings are the common language for multimodal data.